DBMS Unit - 4 (Textbook)

- 📘 Introduction to Transaction Processing
  - 🌐 What is a Transaction Processing System (TPS)?
  - 🧩 What is a *Transaction*?
  - 👤 vs 👥 Single-User and Multiuser Systems
    - 🧍 Single-User DBMS
    - 👨‍👩‍👧‍👦 Multiuser DBMS
  - 🧠 How Can Many Users Run at the Same Time?
    - 🔄 Multiprogramming (Key Idea)
    - ⏱️ Interleaved vs ⚙️ Parallel Processing
    - 🔄 Interleaved Processing
    - ⚙️ Parallel Processing
  - 🗃️ Transactions & Database Items
    - 📦 Database Items
  - 📖 Basic Database Operations
    - 🔹 `read_item(X)`
    - 🔹 `write_item(X)`
  - 🧠 DBMS Buffers & Cache
  - 📊 Read-Set & Write-Set
  - ⚠️ Why Concurrency Control Is Needed
    - 🧠 The Big Idea (in plain words)
  - ❌ 1. Lost Update Problem
    - 💥 What goes wrong?
    - 🧾 What happens?
    - 📌 Example
  - 🧪 2. Temporary Update (Dirty Read) Problem
    - 💥 What goes wrong?
    - 🧾 What happens?
  - 📊 3. Incorrect Summary Problem
    - 💥 What goes wrong?
    - 🧾 What happens?
  - 🔁 4. Unrepeatable Read Problem

- 💥 What goes wrong?
- 🧾 What happens?
- 🔄 21.1.4 Why Recovery Is Needed
  - 🧠 Core Promise of a DBMS
- 🔁 Commit vs Abort (In One Breath)
- 💥 Types of Failures (Why Recovery Exists)
  - ⚙️ 1. System Crash (Very Common)
  - 🧮 2. Transaction or System Errors
  - 🚨 3. Local Errors / Exception Conditions
  - 🔐 4. Concurrency Control Enforcement
  - 💽 5. Disk Failure (Rare, Serious)
  - 🌪️ 6. Physical Catastrophes (Rare, Terrifying)
  - 🧠 Reality Check
- 🧩 Why the Concept of a Transaction Matters

🚦 Transaction and System Concepts

- 🔁 21.2.1 Transaction States & Operations
- 🧾 Key Transaction Operations
  - ▶️ BEGIN_TRANSACTION
  - 📖 READ / ✍️ WRITE
  - ⏹️ END_TRANSACTION
  - ✅ COMMIT_TRANSACTION
  - ❌ ROLLBACK / ABORT
- 🚥 Transaction States (Exam Favorite ⭐)
  - 🔐 Commit Point (Critical Moment)
- 🔄 Rollback & Restart
- 🧾 21.2.2 The System Log
  - 🧠 Why a System Log Is Needed
  - 📂 What Is the System Log?
  - 🧠 Log Buffer (Performance Hack)
- 🧩 Types of Log Records (Exam Gold ⭐)
  - ▶️ 1. Start Record
  - ✍️ 2. Write Record
  - 📖 3. Read Record
  - ✅ 4. Commit Record
  - ❌ 5. Abort Record
- 🔄 Undo & Redo (The Recovery Superpowers)
  - 🔁 UNDO
  - 🔁 REDO

- 📍 Key Assumption
- 🚦 21.2.3 Commit Point of a Transaction
  - 📌 What Is the Commit Point?
  - ✅ Commit Process (Step-by-Step)
  - 💥 If a System Crash Occurs...
  - ⚠️ Force-Writing the Log
- 🔐 Desirable Properties of Transactions (ACID)
  - 🧪 ACID Properties Explained (Simple + Sharp)
    - ⚛️ Atomicity
    - 🧮 Consistency Preservation
    - 🧍 Isolation
    - 🔢 Isolation Levels (Conceptual)
    - 🧱 Durability
- 🧩 Schedules (Histories) of Transactions
  - 🧠 What Is a Schedule?
    - 📌 Key Rules of a Schedule
  - 🔤 Shorthand Notation for Schedules (Exam-Friendly ✍️)
    - ✏️ Example
  - ⚔️ Conflicting Operations (Very Important 🔥)
    - 🔍 Examples of Conflicts
    - 🧠 Intuition
  - ✅ Complete Schedule
  - 🎯 Committed Projection C(S)
    - 📌 Committed Projection `C(S)`
- 🔁 Serializability of Schedules
  - 🧠 Why Serializability?
  - 🧠 Serial vs Non-serial Schedules
    - 🧱 Serial Schedule
    - 🔀 Nonserial Schedule
  - 🚧 Why Serial Schedules Are Impractical
  - 🎯 Expected Correct Result (Running Example)
  - ❌ Why Schedule C Fails
  - 🔐 Serializability (The Gold Standard)
    - 📌 Definition
    - 🧠 Big Picture
  - ⚖️ What Does "Equivalent" Mean?
    - ❌ Result Equivalence (Not Enough)
  - ⚔️ Conflict Equivalence (What We Use)

- 📌 Definition
- 🔥 Reminder: Conflicting Operations
- 🧠 Why Conflict Equivalence Works
- ✅ Conflict-Serializable Schedule
  - 📌 Definition
- 🔍 Applying This to Figure 21.5
  - ✅ Schedule D
  - ❌ Schedule C
- 🧠 Testing for **Conflict Serializability**
  - 🎯 Why Do We Need a Test?
  - 🕸️ Precedence Graph (Serialization Graph)
    - 📌 Graph Definition
  - ⚔️ When Do We Add an Edge?
  - 📜 Algorithm 21.1 — Step by Step (Exam-Perfect)
    - 🪜 Step 1: Create Nodes
    - 🪜 Step 2: Write → Read conflict
    - 🪜 Step 3: Read → Write conflict
    - 🪜 Step 4: Write → Write conflict
    - 🪜 Step 5: Check for Cycles 🚨
  - 🔄 Why Cycles Are Bad
  - 🔍 Interpreting the Graph
  - 📊 Applying to Figure 21.7 (Schedules A–D)
    - (a) Schedule A — Serial
    - (b) Schedule B — Serial
    - (c) Schedule C — ❌ Not Serializable
    - (d) Schedule D — ✅ Serializable
  - 🧠 Key Insight
  - 🔁 Multiple Equivalent Serial Schedules
  - Example Schedules to Practise
  - Answers -
  - 🔄 Characterizing Schedules Based on **Recoverability**
  - 🧠 Why Recoverability Matters
  - ✅ Recoverable Schedules
    - 📌 Definition
    - 🧩 “Reads-from” (Key Phrase)
    - 👍 Why This Is Good
    - ⚠️ Caveat
  - ❌ Nonrecoverable Schedules (Forbidden)

- 🚨 What Goes Wrong
- 🧾 Example (Nonrecoverable)
- 🔁 Cascading Rollback (Still Recoverable, But Painful)
  - 📌 What It Is
  - 🧾 Example (Recoverable, but Cascading)
  - 😬 Why It's Bad
- 🚫 Cascade-less Schedules (Better)
  - 📌 Definition
  - 👍 Benefits
  - ⏳ Trade-off
- 🔒 Strict Schedules (Best for Recovery)
  - 📌 Definition
  - 🧠 Why Strict Is Awesome
  - 🚨 Why Non-Strict Can Break Recovery
- 🧠 The Golden Hierarchy (Remember This!)
  - 🔍 Meaning
- 🔐 Locks in DBMS (The Big Idea)
- 🔐 Binary Locks (DBMS)
  - 🧠 Definition
  - 🔘 Lock States
  - 🔁 Binary Lock Operations
  - 🔄 How Binary Locks Work (Step-by-Step)
  - 📋 Rules for Using Binary Locks (Very Important)
  - 🗂️ Lock Table & Lock Manager
  - ⚠️ Limitations of Binary Locks
- 🔐 Shared / Exclusive (Read–Write) Locks
  - 🧠 Why We Need Them
  - 🧩 Types of Locks
  - ⚙️ Lock Operations
  - 1️⃣ `read_lock(X)`
  - 2️⃣ `write_lock(X)`
  - 3️⃣ `unlock(X)`
  - 🔄 How Shared / Exclusive Locks Work
  - 📖 Read Lock Request
  - ✍️ Write Lock Request
  - 🔁 Lock Conversion
  - 🗂️ Lock Table Structure
  - ⚛️ Atomicity Requirement

📜 Rules for Shared / Exclusive Locks (Exam Gold ⭐)
🔄 Lock Conversion (Upgrading & Downgrading)
⬆️ Upgrade (Read → Write)
⬇️ Downgrade (Write → Read)
⚠️ Important Limitation
🔐 **Guaranteeing Serializability using Two-Phase Locking (2PL)**
🧠 Why Locks Alone Are NOT Enough
🔑 Two-Phase Locking (2PL): The Core Rule
⏱️ The Two Phases
🌱 1. Growing (Expanding) Phase
🍂 2. Shrinking Phase
❌ Why T_1 and T_2 (Figure 21.3) Violate 2PL
✅ Fixing It: 2PL-Compliant Transactions (Figure 21.4)
⚠️ Side Effect of 2PL: Deadlock & Starvation
🧠 The Big Theorem (Exam Favorite ⭐)
💸 Cost of 2PL (Trade-off)
🧬 Variants of Two-Phase Locking
1️⃣ Basic Two-Phase Locking (Basic 2PL)
📌 Definition
🔄 Timeline
✅ Guarantees
❌ Problems
📉 Why It's Weak
🧠 Exam Line
2️⃣ Conservative Two-Phase Locking (Static 2PL)
📌 Definition
🔄 Timeline
✅ Guarantees
❌ Problems
🧠 Why No Deadlock?
🧠 Exam Line
3️⃣ Strict Two-Phase Locking (Strict 2PL) ⭐ MOST IMPORTANT
📌 Definition
🔄 Timeline
✅ Guarantees
❌ Problems
🧠 Why Strict 2PL Is Loved
🧠 Exam Line ⭐

4️⃣ Rigorous Two-Phase Locking (Rigorous 2PL)
📌 Definition
🔄 Timeline
✅ Guarantees
❌ Problems
🧠 Difference from Strict 2PL

📘 Introduction to Transaction Processing

Where databases juggle a hundred hands without dropping a single plate.

🌐 What is a Transaction Processing System (TPS)?

A **Transaction Processing System** is a database system designed to handle:

- 🗄️ **Very large databases**
- 👥 **Hundreds (or thousands) of users at the same time**
- ⚡ **Fast response + high availability**

📌 **Real-world examples**:

- Airline reservation systems ✈️
- Banking & ATM systems 🏦
- Credit card processing 💳
- Online shopping platforms 🛒
- Stock markets 📈

👉 These systems can't afford mistakes. One half-done operation = chaos.

🧩 What is a *Transaction*?

A **transaction** is a **logical unit of work** on a database.

Think of it as:

> "All or nothing. Finish fully, or don't start at all."

A transaction may include:

- 📥 Reading data
- ✏️ Updating data
- ➕ Inserting data
- ❌ Deleting data

🔹 If it **only reads data** → *Read-only transaction*

🔹 If it **updates data** → *Read-write transaction*

Transactions are usually marked by:

```
BEGIN TRANSACTION
   (database operations)
END TRANSACTION
```

👤 vs 👥 Single-User and Multiuser Systems

🧍 Single-User DBMS

- Only **one user at a time**
- Mostly on **personal computers**
- Simple, low risk of conflicts

👨‍👩‍👧‍👦 Multiuser DBMS

- **Many users access the database simultaneously**
- Used in banks, airlines, stock exchanges, supermarkets, etc.
- **Hundreds of transactions run together**

⚠️ This is where problems begin if not controlled properly.

🧠 How Can Many Users Run at the Same Time?

🔄 Multiprogramming (Key Idea)

A computer may have **one CPU**, but it *appears* to do many things at once.

How?

- The CPU rapidly switches between processes
- While one process waits for I/O (disk, network), another runs
- This creates **concurrent execution**

⏱️ Interleaved vs ⚙️ Parallel Processing

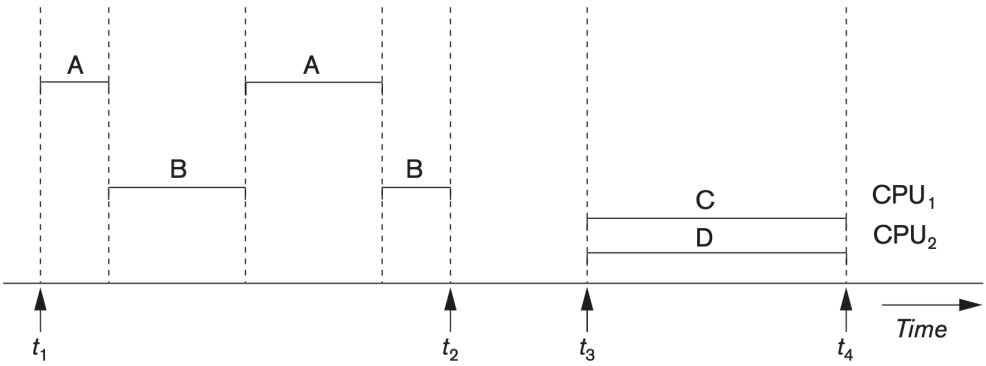


Figure 21.1
Interleaved processing versus parallel processing of concurrent transactions.

🔄 Interleaved Processing

- Single CPU
- One process runs **a bit**, then pauses
- Another process runs
- Looks simultaneous, but isn't truly parallel

⚙️ Parallel Processing

- Multiple CPUs
- Multiple processes run **at the same exact time**

📌 **Important**:

Most **database concurrency theory assumes interleaving**, not parallelism.

🗃️ Transactions & Database Items

📦 Database Items

A database is viewed as a collection of **named data items**.

🔍 A data item can be:

- A full record
- A disk block
- A single attribute (field)

👉 The **size of a data item** is called its **granularity**.

📖 Basic Database Operations

🔹 `read_item(X)`

Reads item `X` from the database into a program variable.

Steps:

1. Find disk block containing `X`
2. Load block into **main memory buffer**
3. Copy `X` into program variable

🔹 `write_item(X)`

Writes program variable `X` back to the database.

Steps:

1. Find disk block containing `X`
2. Load block into memory buffer
3. Update `X` inside the buffer
4. Write buffer back to disk (now or later)

⚠️ **Step 4 actually changes the database on disk**

🧠 DBMS Buffers & Cache

- DBMS keeps **buffers in main memory**

- Each buffer usually holds **one disk block**
- If buffers are full:
  - A **replacement policy** decides which buffer to evict
  - If the buffer was modified → it **must be written back to disk**

🛠️ This coordination is handled by:

- **Recovery manager**
- **Operating system**

📊 Read-Set & Write-Set

For any transaction:

- **Read-set** → items it reads
- **Write-set** → items it writes

📌 Example:

If transaction `T1` reads and updates `X` and `Y`:

- Read-set = `{X, Y}`
- Write-set = `{X, Y}`

(a)

T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

(b)

T_2

read_item(X);
$X := X + M$;
write_item(X);

Figure 21.2
Two sample transactions. (a) Transaction T_1. (b) Transaction T_2.

⚠️ Why Concurrency Control Is Needed

🧠 The Big Idea (in plain words)

In **multiuser databases**, many transactions run **at the same time**.

If their operations are **not controlled**, they can:

- overwrite each other's work,
- read values that were never meant to exist,
- or compute totally wrong results.

👉 **Concurrency control** exists to prevent these logical disasters.

❌ 1. Lost Update Problem

💥 What goes wrong?

Two transactions update the **same data item**, but **one update overwrites the other**.

🧾 What happens?

- T_1 reads X and subtracts `N`
- T_2 reads the **old value of X**
- T_2 writes its result **after T_1**
- 🔥 T_1's update is **lost**

Time	T_1	T_2
	read_item(X); $X := X - N$;	
		read_item(X); $X := X + M$;
	write_item(X); read_item(Y);	
		write_item(X);
	$Y := Y + N$; write_item(Y);	

Item X has an incorrect value because its update by T_1 is *lost* (overwritten).

Figure 21.3
Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

📌 Example

- Initial `X = 80`

- `N = 5`, `M = 4`
- Correct result: `X = 79`
- Actual result (due to lost update): `X = 84` ❌

🧠 **Why it's bad**: The database forgets one transaction ever happened.

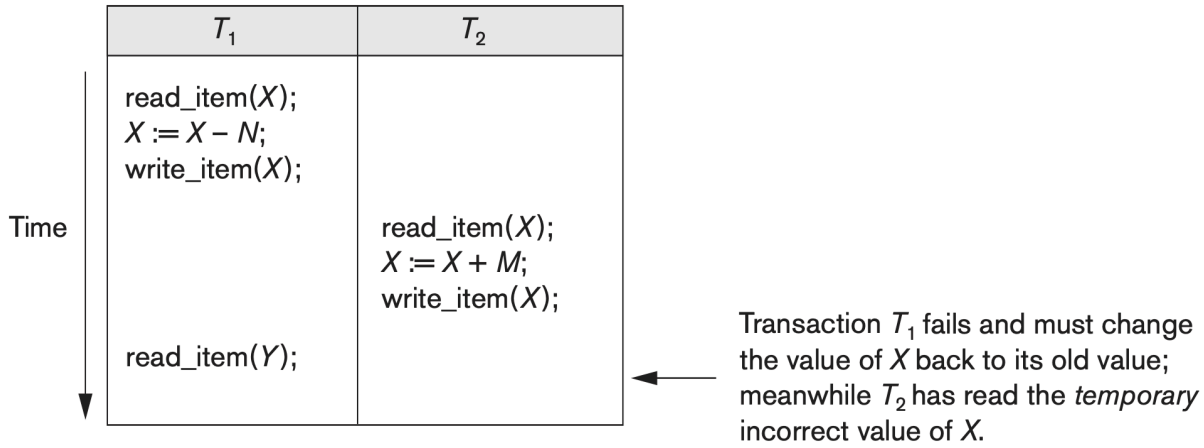
🧪 2. Temporary Update (Dirty Read) Problem

💥 What goes wrong?

A transaction reads **uncommitted data** from another transaction that later **fails**.

🧾 What happens?

- T_1 updates X
- T_2 reads the updated X
- T_1 fails ❌ → X is rolled back
- T_2 has already used **fake data**



📌 The value read by T_2 is called **dirty data**.

🧠 **Why it's bad**: Decisions are made using values that never truly existed.

📊 3. Incorrect Summary Problem

💥 What goes wrong?

A transaction computes an **aggregate (sum, count, etc.)** while another transaction is modifying data.

🧾 What happens?

- T_3 starts summing values
- T_1 subtracts `N` from X
- T_3 reads updated X
- T_3 reads Y **before** `N` is added
- ❌ Final sum is **off by N**

T_1	T_3
	$sum := 0$; read_item(A); $sum := sum + A$; ⋮

read_item(X);
$X := X - N$;
write_item(X);	

	read_item(X);
	$sum := sum + X$;
	read_item(Y);
	$sum := sum + Y$;

read_item(Y);
$Y := Y + N$;
write_item(Y);	

T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

🧠 **Why it's bad**: Reports, analytics, and totals become mathematically wrong.

🔁 4. Unrepeatable Read Problem

💥 What goes wrong?

A transaction reads the **same item twice** and gets **different values**.

🧾 What happens?

- Transaction T reads X
- Another transaction updates X
- T reads X again → different value

📌 Common in:

- seat availability checks
- inventory systems
- financial balances

🧠 **Why it's bad**: The database behaves like a moving target.

🔄 21.1.4 Why Recovery Is Needed

Because databases live in the real world—where things crash, burn, and occasionally spill coffee.

🧠 Core Promise of a DBMS

Whenever a **transaction** runs, the DBMS must guarantee **exactly one** outcome:

✅ **Commit** → *All operations succeed and are permanently stored*

❌ **Abort (Rollback)** → *Nothing happens at all*

There is **no in-between**.

A transaction is a **logical indivisible unit**—half-done is as bad as wrong.

🔁 Commit vs Abort (In One Breath)

- **Committed transaction**

  → Effects are **permanent**, even if the system crashes right after.

- **Aborted transaction**

  → All its effects are **undone**, as if it never existed.

📌 If a transaction fails after some writes → those writes **must be reversed**.

💥 Types of Failures (Why Recovery Exists)

Failures are grouped into **three broad classes**, with six common causes:

⚙️ 1. System Crash (Very Common)

- Hardware, OS, software, or network failure
- Main memory contents may be lost
- Example: sudden power cut ⚡

🧮 2. Transaction or System Errors

- Division by zero
- Integer overflow
- Invalid input parameters
- Logical bugs in code
- User manually cancels the transaction

📌 These are **transaction-level failures**, not DBMS faults.

🚨 3. Local Errors / Exception Conditions

- Data not found
- Business rule violation

  (e.g., insufficient bank balance)

🧠 If **programmed intentionally**, these are **not considered failures**, just controlled aborts.

🔐 4. Concurrency Control Enforcement

- Transaction violates **serializability**
- Deadlock detected 🕸️
- DBMS aborts one or more transactions to recover

📌 Such transactions are **usually restarted automatically**.

💽 5. Disk Failure (Rare, Serious)

- Disk read/write malfunction
- Head crash
- Loss of disk blocks

🌪️ 6. Physical Catastrophes (Rare, Terrifying)

- Fire 🔥
- Flood 🌊
- Theft 🕵️
- Power & cooling failure
- Human mistakes (wrong disk/tape mounted)

🧠 Reality Check

- Failures **1–4** → common, manageable
- Failures **5–6** → rare, but recovery is **heavy-duty**

🧩 Why the Concept of a Transaction Matters

Transactions are the **foundation** for:

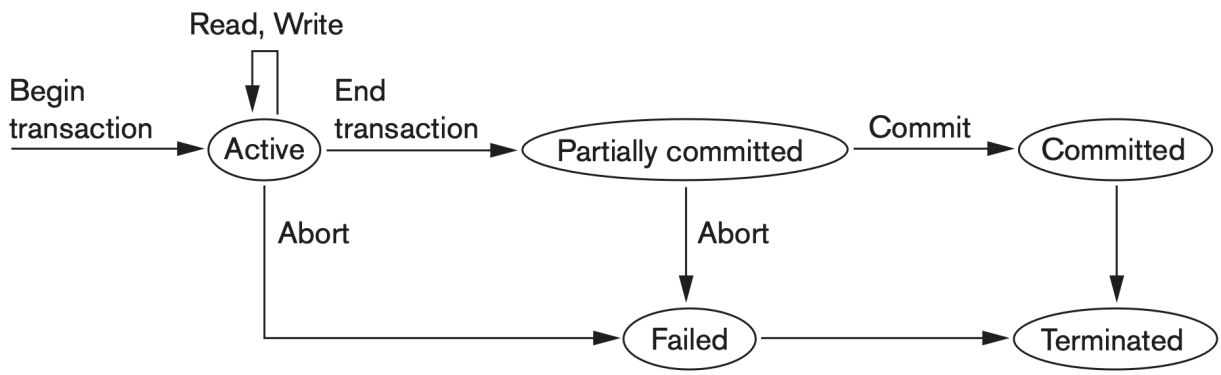
- Concurrency control 🔐
- Recovery mechanisms 🔄
- Atomicity & consistency 🧠

No transactions → no safe recovery.

🚦 Transaction and System Concepts

🔁 21.2.1 Transaction States & Operations

A transaction moves through **well-defined states** during its lifetime



🧾 Key Transaction Operations

▶️ BEGIN_TRANSACTION

- Marks the **start** of a transaction

📖 READ / ✍️ WRITE

- Database access operations
- Executed while transaction is active

⏹️ END_TRANSACTION

- Logical end of transaction execution
- DBMS now decides:
  - Can it commit?
  - Or must it abort?

✅ COMMIT_TRANSACTION

- Transaction **successfully completes**
- All changes become **permanent**

- Will **never be undone**

❌ ROLLBACK / ABORT

- Transaction **fails**
- All effects must be **undone**
- Database restored to previous state

🚦 Transaction States (Exam Favorite ⭐)

State	Meaning
Active	Transaction is executing
Partially Committed	Finished execution, waiting for final checks
Committed	Changes permanently saved
Failed	Transaction cannot continue
Terminated	Transaction leaves the system

🔐 Commit Point (Critical Moment)

- Occurs **after safety checks**
- DBMS ensures:
  - Logs are written
  - Recovery is possible
- After this point → transaction **must survive crashes**

🔄 Rollback & Restart

- Failed transactions:
  - Undo all writes
  - Enter **terminated state**

- They may be:
  - Automatically restarted
  - Or resubmitted later as **new transactions**

🧠 Once terminated, all transaction info is removed from system tables.

🧾 21.2.2 The System Log

The database's black box recorder — it never forgets, even when the system does.

🧠 Why a System Log Is Needed

When failures happen (and they **will**), the DBMS needs a **reliable memory** of:

- what transactions started,
- what data was changed,
- and which transactions finished successfully.

👉 That memory is the **system log**.

📂 What Is the System Log?

- A **sequential, append-only file**
- Stored on **disk** (safe from memory crashes)
- Written **only at the end** (no overwriting)

🛡️ It survives:

- transaction failures
- system crashes

🚨 Except:

- disk failures
- catastrophic failures (fire, theft, etc.)

🧠 Log Buffer (Performance Hack)

- Recent log entries are first written to **main memory buffers**
- When full (or required), buffers are **flushed to disk**
- Disk log is periodically **backed up** to archival storage (tape)

👉 This avoids expensive disk writes for every single log entry.

🧩 Types of Log Records (Exam Gold ⭐)

Each transaction gets a unique **transaction-id (T)**.

▶️ 1. Start Record

```
[start_transaction, T]
```

📌 Transaction T has begun execution.

✍️ 2. Write Record

```
[write_item, T, X, old_value, new_value]
```

📌 Transaction T changed item X

🧠 This is the **most important** log record for recovery.

📖 3. Read Record

```
[read_item, T, X]
```

📌 Transaction T read item X

⚠️ Usually **not required** for recovery

✔️ Used for **auditing** or special protocols

✅ 4. Commit Record

```
[commit, T]
```

📌 Transaction T completed successfully

✔️ Changes are now permanent

❌ 5. Abort Record

```
[abort, T]
```

📌 Transaction T failed and must be undone

🔄 Undo & Redo (The Recovery Superpowers)

🔁 UNDO

Used when:

- transaction **did not commit**
- system crashes mid-execution

How?

- Scan log **backwards**
- Restore each item to its **old_value**

🔁 REDO

Used when:

- transaction **committed**
- but changes weren't written to disk yet

How?

- Scan log **forward**
- Reapply each **new_value**

📌 This is why the log stores **before and after values**.

📍 Key Assumption

> All permanent database changes happen inside transactions.

So recovery =

✔️ undo uncommitted transactions

✔️ redo committed ones

🚦 21.2.3 Commit Point of a Transaction

The moment a transaction becomes immortal.

📌 What Is the Commit Point?

A transaction **reaches its commit point** when:

- all READ and WRITE operations finish successfully
- **all WRITE operations are recorded in the log**
- log entries are safely written to disk

Only **after this point** can the transaction commit.

✅ Commit Process (Step-by-Step)

1. Transaction finishes execution
2. Log records are **force-written to disk**
3. `[commit, T]` is written to the log
4. Transaction is officially **committed**

🧠 After this → its effects **must survive crashes**

💥 If a System Crash Occurs...

During recovery:

- Transactions with:

```
[start_transaction, T]
```

but **no** `[commit, T]` → ❌ **ROLLBACK**

- Transactions **with** `[commit, T]` → 🔁 **REDO if needed**

⚠️ Force-Writing the Log

Before committing:

- Any log entries still in memory **must be flushed to disk**
- This ensures recovery info is never lost

📌 This is called **force-writing the log buffer**

🔐 Desirable Properties of Transactions (ACID)

Transactions must obey **ACID** — the sacred contract of databases.

🧪 ACID Properties Explained (Simple + Sharp)

Property	Meaning
Atomicity	All or nothing
Consistency	Valid state → valid state
Isolation	Acts like it's alone
Durability	Once committed, forever

⚛️ Atomicity

- Transaction is **indivisible**
- Partial execution is **not allowed**
- Fail → undo everything

- **A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.**

🧠 Enforced by: **Recovery subsystem**

🧮 Consistency Preservation

- Database constraints must always hold
- Programmer + integrity rules ensure this
- **A correct execution of the transaction must take the database from one consistent state to another.**

📌 If DB is consistent before → consistent after (if no interference)

🧍 Isolation

- Transactions must **not see each other's intermediate results until** they are **committed**.
- Appears as if transactions run **one by one**

🧠 Enforced by: **Concurrency control subsystem**

🔢 Isolation Levels (Conceptual)

Level	Guarantees
0	No dirty overwrites
1	No lost updates
2	No dirty reads
3	Repeatable reads (true isolation)

🧱 Durability

- Once committed → changes **never disappear**
- Even after crashes

🧠 Enforced by: **Recovery subsystem + logging**

🧩 Schedules (Histories) of Transactions

The choreography of concurrent transactions — who moves when, and why it matters.

🧠 What Is a Schedule?

A **schedule (or history) S** is the **global order** in which operations from multiple transactions are executed.

Think of it as:

> One timeline that merges many transaction timelines.

📌 Key Rules of a Schedule

For transactions `T₁, T₂, …, Tₙ`:

1. Operations **from different transactions may interleave**
2. Operations **within the same transaction must keep their original order**
3. The order in a schedule is a **total order**

   → for any two operations, one happens first

📌 (Partial orders exist in theory, but DBMS assumes **total order**.)

🔤 Shorthand Notation for Schedules (Exam-Friendly ✍️)

Symbol	Meaning
b_i	begin_transaction(T_i)
$r_i(X)$	read_item(X) by T_i
$w_i(X)$	write_item(X) by T_i
e_i	end_transaction(T_i)
c_i	commit(T_i)
a_i	abort(T_i)

✏️ Example

Time	T_1	T_2
	read_item(X);	
	$X := X - N$;	
		read_item(X);
		$X := X + M$;
	write_item(X);	
	read_item(Y);	
		write_item(X);
	$Y := Y + N$;	
	write_item(Y);	

From Figure 21.3(a):

```
Sa:r1(X);r2(X);w1(X);r1(Y);w2(X);w1(Y);
```

Time	T_1	T_2
	read_item(X);	
	$X := X - N$;	
	write_item(X);	
		read_item(X);
		$X := X + M$;
		write_item(X);
	read_item(Y);	

Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X.

From Figure 21.3(b):

```
Sb:r1(X);w1(X);r2(X);w2(X);r1(Y); a1;
```

⚔️ Conflicting Operations (Very Important 🔥)

Two operations **conflict** if **all three** hold:

1. Belong to **different transactions**
2. Access the **same data item X**
3. **At least one is a write**

🔍 Examples of Conflicts

- ✅ `r1(X)` ↔ `w2(X)` → **read–write conflict**
- ✅ `w1(X)` ↔ `w2(X)` → **write–write conflict**
- ❌ `r1(X)` ↔ `r2(X)` → *no conflict*
- ❌ `w2(X)` ↔ `w1(Y)` → *different items*
- ❌ `r1(X)` ↔ `w1(X)` → *same transaction*

🧠 Intuition

Two operations conflict **if swapping their order changes the result**.

- **Read–Write conflict**

  → changes what value is read

- **Write–Write conflict**

  → changes the final stored value

✅ Complete Schedule

A schedule **S** is called **complete** if:

1. It contains **all operations** of `T₁, T₂, …, Tₙ`
2. Each transaction ends with **commit or abort**
3. Relative order of operations **within a transaction is preserved**
4. For **every conflicting pair**, the order is defined

📌 At the end of a complete schedule:

- No active transactions remain

🎯 Committed Projection C(S)

Since real systems never stop accepting transactions:

👉 We focus on **committed transactions only**.

📌 Committed Projection `C(S)`

- Contains **only operations of committed transactions**
- Ignores aborted or active ones
- Used for **recovery & correctness analysis**

🔁 Serializability of Schedules

Concurrency is allowed — chaos is not.

🧠 Why Serializability?

If transactions run **one after another**, correctness is guaranteed.

Two possible **serial schedules** for T_1 and T_2:

1. $T_1 \rightarrow T_2$
2. $T_2 \rightarrow T_1$

These are shown in **Figure (a) and (b)**.

(a)

T_1	T_2

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);	

	read_item(X);
	$X := X + M$;
	write_item(X);

Time

Schedule A

(b)

T_1	T_2

	read_item(X);
	$X := X + M$;
	write_item(X);

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);	

Time

Schedule B

🧠 Serial vs Non-serial Schedules

🧱 Serial Schedule

- All operations of **one transaction execute consecutively**
- No interleaving at all
- Only **one active transaction** at a time

📌 Examples:

- Schedule A → `T₁ → T₂`
- Schedule B → `T₂ → T₁`

✔️ Always correct

❌ Practically inefficient

🔀 Nonserial Schedule

- Operations of multiple transactions **interleave**
- Multiple active transactions at once
- High performance ⚡
- Risk of incorrect results ❌

(c)

T_1	T_2
read_item(X);	
$X := X - N$;	
	read_item(X);
	$X := X + M$;
write_item(X);	
read_item(Y);	
	write_item(X);
$Y := Y + N$;	
write_item(Y);	

Time

Schedule C

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
	read_item(X);
	$X := X + M$;
	write_item(X);
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

Time

Schedule D

📌 Examples:

- Schedule C ❌ (incorrect)
- Schedule D ✅ (correct)

🚧 Why Serial Schedules Are Impractical

- CPU sits idle during I/O waits
- Long transactions block short ones
- Throughput collapses under load

👉 **We want concurrency, but with correctness.**

🎯 Expected Correct Result (Running Example)

Given:

- Initial values: `X = 90`, `Y = 90`
- `N = 3`, `M = 2`

After executing `T₁` and `T₂`:

- `X = 89`
- `Y = 93`

✔️ Schedule A → correct

✔️ Schedule B → correct

❌ Schedule C → `X = 92` (highlight: lost update)

✔️ Schedule D → correct

❌ Why Schedule C Fails

- `T₂` reads **old X**
- `T₁`'s update is overwritten
- Classic **lost update problem**

🧠 Nonserial ≠ incorrect

🧠 Some nonserial schedules are safe

🔐 Serializability (The Gold Standard)

📌 Definition

A schedule **S** is **serializable** if:

> It is equivalent to some serial schedule of the same transactions.

✔️ Equivalent = same logical effect

✔️ Therefore correct

🧠 Big Picture

All nonserial schedules split into:

- ✅ **Serializable** → safe
- ❌ **Non-serializable** → unsafe

⚖️ What Does "Equivalent" Mean?

S_1	S_2

read_item(X);
$X := X + 10$;
write_item(X);
	read_item(X);
	$X := X * 1.1$;
	write_item (X);

❌ Result Equivalence (Not Enough)

Two schedules are result equivalent if they:

- Produce the **same final database state**

🚨 Problem:

- Same result can happen **by coincidence**
- Depends on **initial values**
- Not reliable

📌 Example (Figure 21.6):

- Schedules look equal for `X = 100`
- Fail for other values

👉 **Result equivalence is unsafe**

⚔️ Conflict Equivalence (What We Use)

📌 Definition

Two schedules are **conflict equivalent** if:

> The order of every pair of conflicting operations is the same in both schedules.

🔥 Reminder: Conflicting Operations

Two operations conflict if:

1. Different transactions

2. Same data item
3. At least one is a write

🧠 Why Conflict Equivalence Works

- Changing order of conflicting ops ⇒ different outcome
- Same order ⇒ same effect

✔️ Safe

✔️ Independent of data values

✔️ Practical to test

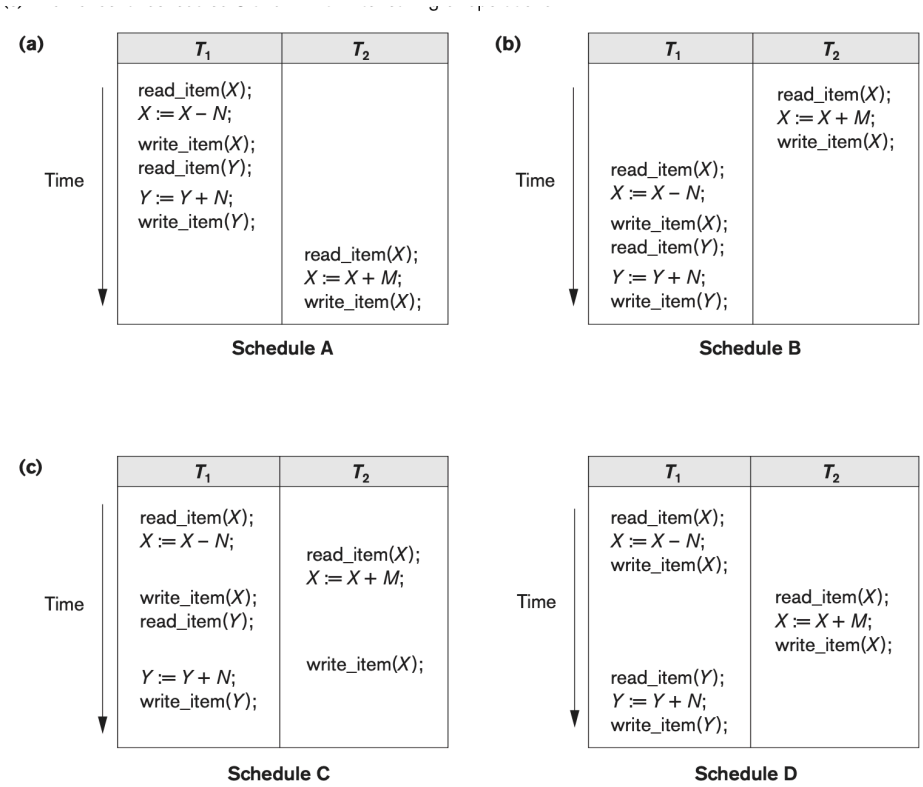
✅ Conflict-Serializable Schedule

📌 Definition

A schedule is **conflict serializable** if:

> It is conflict equivalent to some serial schedule

🔍 Applying This to Figure 21.5



✅ Schedule D

- Conflicting ops occur in same order as **Schedule A**
- Nonconflicting ops can be rearranged
- Equivalent to serial `T₁ → T₂`

✔️ **Conflict-serializable**

❌ Schedule C

- `r₂(X)` ↔ `w₁(X)` conflict
- `w₁(X)` ↔ `w₂(X)` conflict
- Cannot reorder without breaking conflicts

❌ **Not serializable**

🧠 Testing for Conflict Serializability

When schedules get suspicious, we interrogate them with graphs.

🎯 Why Do We Need a Test?

In practice:

- DBMS **does not** test every schedule for serializability
- Instead, it enforces **protocols** (like locking) that *guarantee* serializability

But for **understanding** (and exams 😌), we use a **graph-based test**.

🕸️ Precedence Graph (Serialization Graph)

A **precedence graph** is a **directed graph** that captures **ordering constraints** between transactions caused by **conflicting operations**.

📌 Graph Definition

- **Nodes (N)** → transactions `{T₁, T₂, …, Tₙ}`
- **Edges (E)** → ordering constraints `(Tᵢ → Tⱼ)`

👉 An edge means:

T_i must occur before T_j in any equivalent serial schedule

⚔️ When Do We Add an Edge?

Look **only** at `read_item` and `write_item` operations.

Add edge **(T_i → T_j)** if **T_i's operation appears before a conflicting operation of T_j**.

📜 Algorithm 21.1 — Step by Step (Exam-Perfect)

🪜 Step 1: Create Nodes

For every transaction `Tᵢ` in schedule `S`

→ create a node `Tᵢ`

🪜 Step 2: Write → Read conflict

If:

```
wᵢ(X) appears before rⱼ(X)
```

➡️ Add edge:

```
Tᵢ → Tⱼ
```

🪜 Step 3: Read → Write conflict

If:

```
rᵢ(X) appears before wⱼ(X)
```

➡️ Add edge:

```
Tᵢ → Tⱼ
```

🪜 Step 4: Write → Write conflict

If:

```
wᵢ(X) appears before wⱼ(X)
```

➡️ Add edge:

```
Tᵢ → Tⱼ
```

🪜 Step 5: Check for Cycles 🚨

- ❌ **Cycle present → NOT serializable**
- ✅ **No cycle → Conflict-serializable**

🔄 Why Cycles Are Bad

A **cycle** means:

```
T₁ → T₂ → … → T₁
```

🧠 Interpretation:

- T_1 must happen before T_2
- T_2 must happen before T_1
- ❌ Impossible in a serial order

🔍 Interpreting the Graph

- **Edge $T_i \rightarrow T_j$**

  → T_i must precede T_j in any equivalent serial schedule

- **No incoming edges**

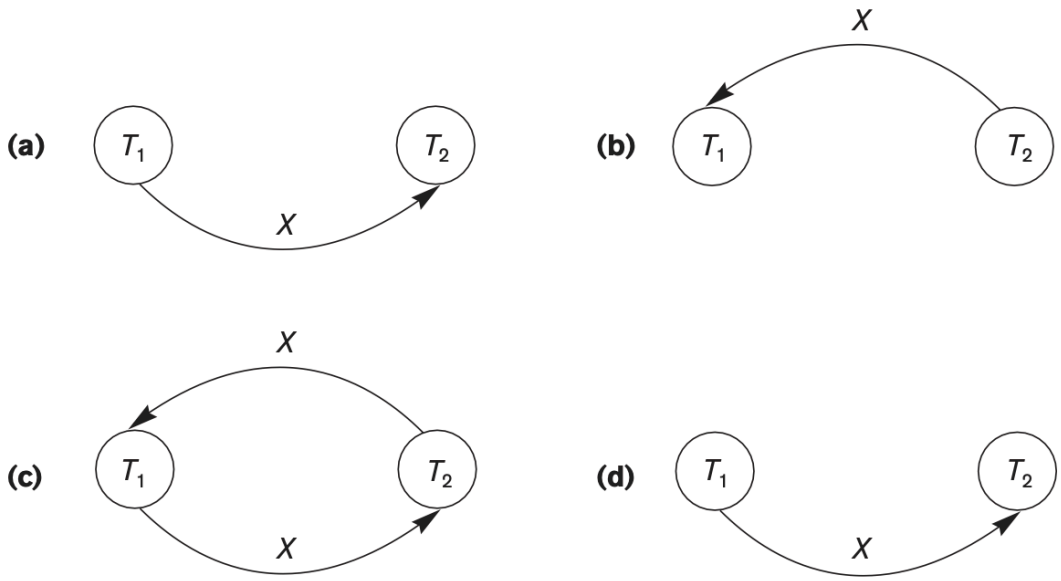
  → transaction can safely go **first**

- **Topological sorting**

  → gives an equivalent serial order

📌 (This is called **topological ordering of an acyclic graph**.)

📊 Applying to Figure 21.7 (Schedules A–D)



(a) Schedule A — Serial

- Graph: `T₁ → T₂`
- No cycle ✅
- Serializable (already serial)

(b) Schedule B — Serial

- Graph: `T₂ → T₁`
- No cycle ✅
- Serializable

(c) Schedule C — ❌ Not Serializable

- Graph contains:

```
T₁ → T₂ → T₁
```

- Cycle detected ❌
- Lost update case

(d) Schedule D — ✅ Serializable

- Graph: `$T_1 \rightarrow T_2$`
- No cycle
- Equivalent to **Schedule A**

🧠 Key Insight

> Conflict serializability ⇔ acyclic precedence graph

That's the entire test.

🔁 Multiple Equivalent Serial Schedules

If the graph:

- has **no cycle**
- and has **multiple valid topological orders**

👉 Then **multiple serial schedules** are equivalent.

📌 Rule to find one:

1. Pick a node with **no incoming edges**
2. Remove it
3. Repeat

Example Schedules to Practise

(b)

Transaction T_1	Transaction T_2	Transaction T_3
	read_item(Z);	
	read_item(Y);	
	write_item(Y);	
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(X);	
read_item(Y);		
write_item(Y);	write_item(X);	

Time

Schedule E

(c)

Transaction T_1	Transaction T_2	Transaction T_3
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(Z);	
read_item(Y);		
write_item(Y);	read_item(Y);	
	write_item(Y);	
	read_item(X);	
	write_item(X);	

Time

Schedule F

Answers -

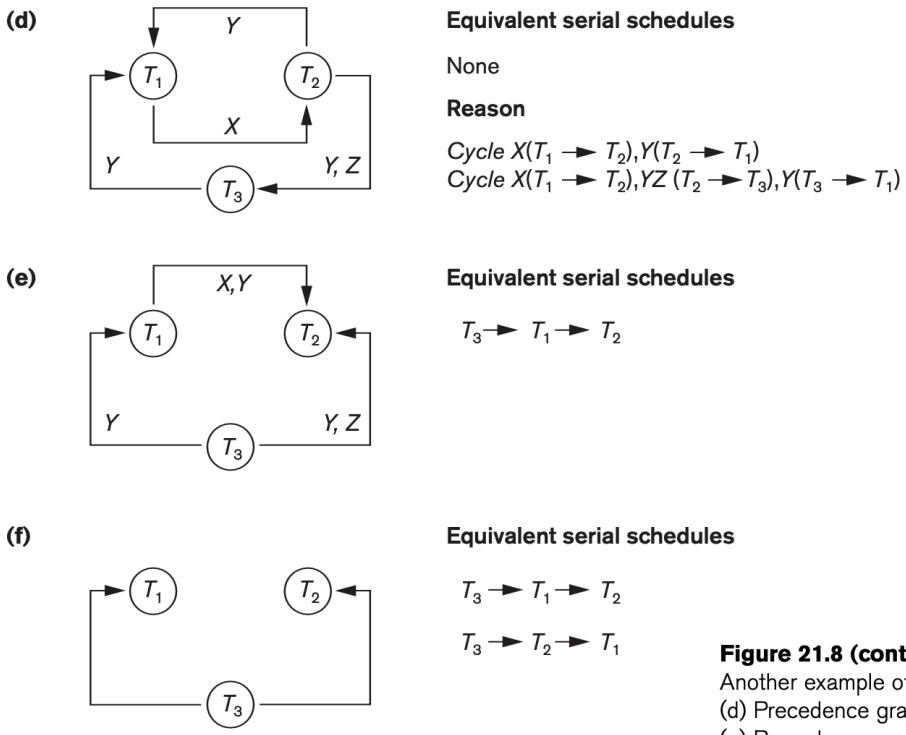


Figure 21.8 (continued)
Another example of serializability testing.
(d) Precedence graph for schedule E.
(e) Precedence graph for schedule F.
(f) Precedence graph with two equivalent serial schedules.

🔄 Characterizing Schedules Based on Recoverability

*Serializability keeps results correct; **recoverability** makes sure crashes don't turn correctness into regret.*

🧠 Why Recoverability Matters

After a failure, the DBMS must **recover cleanly**:

- ❌ Never roll back a **committed** transaction
- 🔁 Roll back only what's safe and necessary
- ⚡ Avoid chain-reaction aborts when possible

So we classify schedules by **how hard recovery becomes**.

✅ Recoverable Schedules

📌 Definition

A schedule **S** is **recoverable** if:

> No transaction commits until all transactions it read from have committed.

🧩 "Reads-from" (Key Phrase)

Transaction **Tj reads from Ti** if:

- `Ti` **writes X**
- Later `Tj` **reads X**
- `Ti` has **not aborted** before the read
- No other write to X intervenes (unless aborted)

👍 Why This Is Good

- Once **Tj commits**, it will **never need rollback**
- Durability is preserved

⚠️ Caveat

Recoverable ≠ correct (serializability handles correctness).

Recoverable only promises **safe recovery**.

❌ Nonrecoverable Schedules (Forbidden)

🚨 What Goes Wrong

- `T2` reads data written by `T1`
- `T2` **commits**
- `T1` **aborts later**

👉 Now `T2` has committed **dirty data** and must be rolled back — **illegal**.

🧾 Example (Nonrecoverable)

```
Sc:r1(X);w1(X);r2(X); ...; c2; a1;
```

❌ `T2` commits before `T1` commits → not recoverable

🔁 Cascading Rollback (Still Recoverable, But Painful)

📌 What It Is

If an uncommitted transaction aborts, **every transaction that read from it must also abort** — and so on.

🧾 Example (Recoverable, but Cascading)

```
Se:r1(X);w1(X);r2(X); ...; a1; a2;
```

- `T2` read from `T1`
- `T1` aborts → `T2` must abort too

😬 Why It's Bad

- One abort → many aborts
- Expensive, slow recovery

🚫 Cascade-less Schedules (Better)

📌 Definition

A schedule is **cascade-less** if:

> Transactions read only data written by committed transactions.

👍 Benefits

- **No cascading rollback**
- Recovery is simpler and faster

⏳ Trade-off

- Reads may be **delayed** until writers commit

🔒 Strict Schedules (Best for Recovery)

📌 Definition

A schedule is **strict** if:

> A transaction can neither read nor write X until the last transaction that wrote X commits or aborts.

🧠 Why Strict Is Awesome

- Recovery is **trivial**
- Undo = restore **before-image (BFIM)** from the log
- No dirty reads, no dirty writes

🚨 Why Non-Strict Can Break Recovery

```
Sf:w1(X,5); w2(X,8); a1;
```

- Original `X = 9`
- `T1` aborts → restore BFIM to `9`
- But `T2` already wrote `8` ❌ (corruption risk)

Strict schedules **prevent this entirely**.

🧠 The Golden Hierarchy (Remember This!)

```
Strict ⊂ Cascadeless ⊂ Recoverable ⊂ All Schedules
```

🔍 Meaning

- **All strict** schedules are cascadeless

- **All cascadeless** schedules are recoverable
- Reverse is **not** true

🔐 Locks in DBMS (The Big Idea)

A **lock** is a *permission flag* attached to a **data item** (X, Y, ...) that controls:

- **who can read**
- **who can write**
- **who must wait**

👉 Locks are how DBMS prevents:

- dirty reads
- dirty writes
- lost updates
- chaos in general

🔐 Binary Locks (DBMS)

🧠 Definition

A **binary lock** is the simplest form of locking used for concurrency control.

Each data item can be in **only one of two states**.

🔘 Lock States

State	Value	Meaning
Unlocked	`0`	Data item is free and can be accessed
Locked	`1`	Data item is currently in use by a transaction

📌 **Each database item X has its own lock** `LOCK(X)`

🔁 Binary Lock Operations

```
lock_item(X):
B:   if LOCK(X) = 0                (*item is unlocked*)
         then LOCK(X) ←1           (*lock the item*)
     else
         begin
         wait (until LOCK(X) = 0
             and the lock manager wakes up the transaction);
         go to B
         end;
unlock_item(X):
     LOCK(X) ← 0;                  (* unlock the item *)
     if any transactions are waiting
         then wakeup one of the waiting transactions;
```

🔄 How Binary Locks Work (Step-by-Step)

1. Transaction requests `lock_item(X)`
2. If `LOCK(X) = 0`

   → lock is granted (`LOCK(X) = 1`)
3. If `LOCK(X) = 1`

   → transaction waits
4. Transaction reads/writes item X
5. Transaction executes `unlock_item(X)`
6. Waiting transaction (if any) is allowed to proceed

📌 Locking and unlocking operations must be **atomic**

(no interruption allowed once they start)

📋 Rules for Using Binary Locks (Very Important)

1. A transaction **must lock** an item before reading or writing it
2. A transaction **must unlock** the item after finishing with it
3. A transaction **cannot lock** an item it already holds
4. A transaction **cannot unlock** an item it does not hold

These rules are enforced by the **lock manager**.

🗂️ Lock Table & Lock Manager

- The DBMS maintains a **lock table**
- Contains entries for **currently locked items**
- Helps track:
  - which transaction holds which lock
  - which transactions are waiting

👉 The **lock manager**:

- grants locks
- makes transactions wait
- wakes them up when locks are released

⚠️ Limitations of Binary Locks

- Only **one transaction** can access an item at a time
- Even **multiple readers are not allowed**
- Leads to **low concurrency**

📌 Because of this, binary locks are:

- easy to understand
- but **too restrictive**
- rarely used in real DBMS

🔐 Shared / Exclusive (Read–Write) Locks

🧠 Why We Need Them

Binary locks are too restrictive:

- They allow **only one transaction** to access an item.
- Even **multiple readers** are blocked.

👉 **Shared/Exclusive locks** improve concurrency by distinguishing **reading** from **writing**.

🧩 Types of Locks

Lock Type	Also Called	Purpose
Shared Lock (S)	Read lock	Allows **multiple readers**
Exclusive Lock (X)	Write lock	Allows **one writer only**

📌 Rule of thumb:

- **Many readers → OK**
- **Any writer → must be alone**

⚙️ Lock Operations

1️⃣ `read_lock(X)`

- Grants **shared access** to item X
- Multiple transactions can hold this lock at the same time

2️⃣ `write_lock(X)`

- Grants **exclusive access** to item X
- No other transaction can read or write X

3️⃣ `unlock(X)`

- Releases the lock on X

- Allows waiting transactions to proceed

🔄 How Shared / Exclusive Locks Work

📖 Read Lock Request

- If X is **unlocked** → grant read lock
- If X is **read-locked** → grant another read lock
- If X is **write-locked** → transaction **waits**

✍️ Write Lock Request

- If X is **unlocked** → grant write lock
- Otherwise → transaction **waits**

🔁 Lock Conversion

- **Read → Write (Upgrade)**

  ✔️ Allowed **only if** the transaction is the **only reader**
- **Write → Read (Downgrade)**

  ✔️ Allowed to increase concurrency

🗂️ Lock Table Structure

Each lock table entry stores:

1. **Data item name**
2. **Lock status** (`read-locked` / `write-locked`)
3. **Number of read locks**
4. **Transaction ID(s)** holding the lock

📌 Managed by the **lock manager** inside the DBMS.

⚛️ Atomicity Requirement

All locking operations must be **atomic**:

- Once a lock/unlock starts, it **cannot be interrupted**
- Prevents race conditions during lock handling

📜 Rules for Shared / Exclusive Locks (Exam Gold ⭐)

1. A transaction **must request a lock** before reading or writing an item
2. A transaction **must request a write lock** before modifying an item
3. A transaction **must release locks** after finishing its operations
4. A transaction **cannot request a lock it already holds**
    - (may be relaxed for lock conversion)
5. A transaction **cannot unlock an item it does not hold**

🔄 Lock Conversion (Upgrading & Downgrading)

⬆️ Upgrade (Read → Write)

- Allowed **only if no other transaction** holds a read lock
- Otherwise → transaction waits

⬇️ Downgrade (Write → Read)

- Writer gives up exclusivity
- Other readers may now proceed

📌 Requires storing **transaction IDs** in the lock table.

⚠️ Important Limitation

> **Shared/Exclusive locks alone do NOT guarantee serializability.**

If locks are:

- acquired correctly

- but **released too early**

👉 Non-serializable schedules can still occur.

🔐 This is why **Two-Phase Locking (2PL)** is required.

🔐 Guaranteeing Serializability using Two-Phase Locking (2PL)

🧠 Why Locks Alone Are NOT Enough

Even with **read/write locks**, you can still get:

- ❌ non-serializable schedules
- ❌ wrong final values

👉 **Problem:** transactions may **unlock too early**, letting others sneak in.

This is exactly what happens in **Figure 21.3(c)**

(a)

T_1	T_2
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
unlock(Y);	unlock(X);
write_lock(X);	write_lock(Y);
read_item(X);	read_item(Y);
$X := X + Y$;	$Y := X + Y$;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

(b) Initial values: X=20, Y=30

Result serial schedule T_1 followed by T_2: X=50, Y=80

Result of serial schedule T_2 followed by T_1: X=70, Y=50

(c)

Time ↓

T_1	T_2
read_lock(Y);	
read_item(Y);	
unlock(Y);	
	read_lock(X);
	read_item(X);
	unlock(X);
	write_lock(Y);
	read_item(Y);
	$Y := X + Y$;
	write_item(Y);
	unlock(Y);
write_lock(X);	
read_item(X);	
$X := X + Y$;	
write_item(X);	
unlock(X);	

Result of schedule S: X=50, Y=50 (nonserializable)

Figure 21.3
Transactions that do not obey two-phase locking. (a) Two transactions T_1 and T_2. (b) Results of possible serial schedules of T_1 and T_2. (c) A nonserializable schedule S that uses locks.

→ locks are used

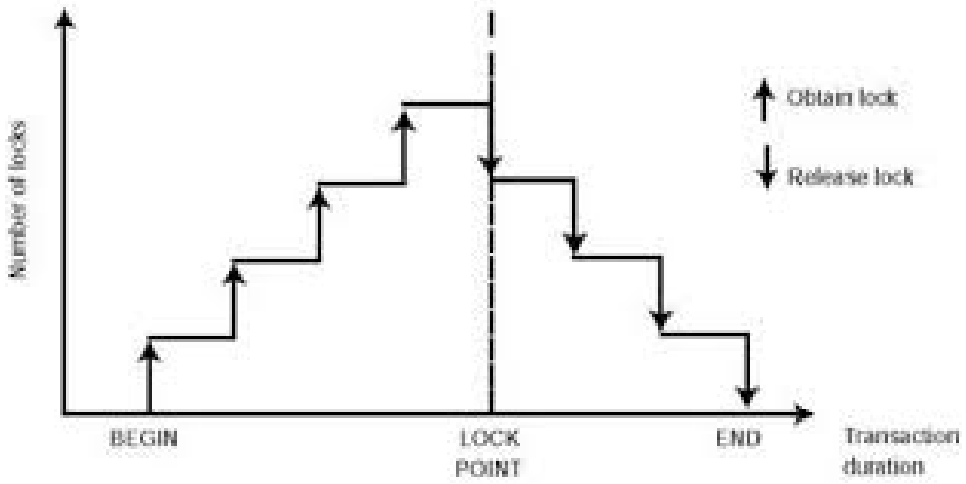
→ but schedule is **nonserializable**

🔑 Two-Phase Locking (2PL): The Core Rule

> A transaction follows **2PL** if **all lock operations happen before the first unlock**.

Once you unlock **anything**, you can **never lock again**.

⏱️ The Two Phases



🌱 1. Growing (Expanding) Phase

- ✅ Can **acquire** locks (read/write)
- ❌ Cannot release any lock
- Lock upgrades (S → X) allowed here

🍂 2. Shrinking Phase

- ✅ Can **release** locks
- ❌ Cannot acquire new locks
- Lock downgrades (X → S) allowed here

📌 The **first unlock** marks the boundary.

❌ Why T_1 and T_2 (Figure 21.3) Violate 2PL

T_1	T_2
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
unlock(Y);	unlock(X);
write_lock(X);	write_lock(Y);
read_item(X);	read_item(Y);
$X := X + Y$;	$Y := X + Y$;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

In **T_1**:

```
read_lock(Y)
read_item(Y)
unlock(Y)       ❌ unlock happens
write_lock(X)   ❌ new lock after unlock → violation
```

Same mistake in **T_2**.

👉 Result: **non-serializable schedule**, even though locks were used.

✅ Fixing It: 2PL-Compliant Transactions (Figure 21.4)

Rewritten transactions **T_1' and T_2'**:

T_1'	T_2'
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
write_lock(X);	write_lock(Y);
unlock(Y)	unlock(X)
read_item(X);	read_item(Y);
$X := X + Y$;	$Y := X + Y$;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

- All locks are taken **before any unlock**
- Unlocking happens only at the end
- Non-serializable schedule is now **impossible**

📌 Result:

✔️ Schedule is **guaranteed serializable**

⚠️ Side Effect of 2PL: Deadlock & Starvation

Example:

- T_1' locks **Y**, wants **X**
- T_2' locks **X**, wants **Y**

Both wait forever 😵

👉 **2PL guarantees serializability, NOT deadlock freedom**

🧠 The Big Theorem (Exam Favorite ⭐)

> **If every transaction follows Two-Phase Locking, every possible schedule is conflict-serializable.**

So:

- No need to test serializability
- The protocol enforces it automatically

💸 Cost of 2PL (Trade-off)

2PL may:

- reduce concurrency
- force transactions to hold locks longer than needed
- delay other transactions unnecessarily

📌 This is the **price paid for guaranteed correctness**.

🧬 Variants of Two-Phase Locking

1️⃣ Basic Two-Phase Locking (Basic 2PL)

📌 Definition

A transaction:

- can acquire locks in the **growing phase**
- can release locks in the **shrinking phase**
- cannot acquire new locks after releasing one

🔄 Timeline

```
lock(A) → lock(B) → unlock(A) → unlock(B)
```

✔️ All locks before first unlock

✔️ Follows 2PL

✅ Guarantees

- ✔️ **Conflict serializability**

❌ Problems

- ❌ **Dirty reads possible**
- ❌ **Cascading rollbacks possible**
- ❌ **Deadlocks possible**
- ❌ Not strict → recovery is harder

📉 Why It's Weak

A transaction may:

- write X
- unlock X
- later abort

  → other transactions may already have read dirty data

🧠 Exam Line

> **Basic 2PL guarantees serializability but not recoverability or strictness.**

2️⃣ Conservative Two-Phase Locking (Static 2PL)

📌 Definition

A transaction:

- **locks ALL required items before it starts**
- if any lock is unavailable → locks nothing and waits

📌 Read-set + Write-set must be **known in advance**

🔄 Timeline

```
lock(A),lock(B),lock(C)
→ execute transaction
→unlock(A),unlock(B),unlock(C)
```

No interleaving during locking.

✅ Guarantees

- ✔️ **Conflict serializability**
- ✔️ **Deadlock-free**
- ✔️ **Recoverable**
- ✔️ **Cascadeless**

❌ Problems

- ❌ Very **low concurrency**
- ❌ **Impractical** (read/write set often unknown)

🧠 Why No Deadlock?

Deadlock needs **partial locking**.

Here, locking is **all-or-nothing**, so circular waits cannot form.

🧠 Exam Line

> **Conservative 2PL avoids deadlocks by prelocking all data items before execution.**

3️⃣ Strict Two-Phase Locking (Strict 2PL) ⭐ MOST IMPORTANT

📌 Definition

A transaction:

- follows **basic 2PL**
- **holds all write (exclusive) locks until commit or abort**

🔓 Read locks *may* be released earlier

🔒 Write locks are held till the end

🔄 Timeline

```
lock S(A) → lock X(B) → unlock S(A)
→ COMMIT
→ unlock X(B)
```

✅ Guarantees

- ✔️ **Conflict serializability**
- ✔️ **Recoverable schedules**
- ✔️ **No dirty reads**
- ✔️ **No cascading rollback**
- ✔️ **Strict schedules**

❌ Problems

- ❌ **Deadlocks still possible**
- ❌ Slightly reduced concurrency

🧠 Why Strict 2PL Is Loved

Recovery becomes **simple**:

- Undo = restore before-image
- Redo = apply after-image

👉 **This is what real DBMSs use**

🧠 Exam Line ⭐

> **Strict 2PL guarantees serializability and strict schedules by holding all write locks until commit or abort.**

4️⃣ Rigorous Two-Phase Locking (Rigorous 2PL)

📌 Definition

A transaction:

- follows 2PL
- **holds ALL locks (read + write) until commit or abort**

Nothing is released early.

🔄 Timeline

```
lockS(A) → lockX(B)
→ execute transaction
→ COMMIT
→unlock(A),unlock(B)
```

✅ Guarantees

- ✔️ **Conflict serializability**
- ✔️ **Strict schedules**
- ✔️ **Cascadeless**
- ✔️ **Simplest recovery**
- ✔️ **Commit order = serialization order**

❌ Problems

- ❌ **Lowest concurrency**

- ❌ **Deadlocks possible**

🧠 Difference from Strict 2PL

Feature	Strict 2PL	Rigorous 2PL
Read locks released early	✔️	❌
Write locks held till commit	✔️	✔️
Simpler recovery	✔️	✔️✔️
Concurrency	Higher	Lower